JOURNAL OF CHEMOMETRICS, VOL. 11, 73–85 (1997)

IMPROVED PLS ALGORITHMS

BHUPINDER. S. DAYAL* AND JOHN F. MACGREGOR‡
Department of Chemical Engineering, McMaster University, Hamilton, ON., L8S-4L7, Canada

SUMMARY

In this paper a proof is given that only one of either the **X**- or the **Y**-matrix in PLS algorithms needs to be deflated during the sequential process of computing latent vectors. With the aid of this proof the original kernel algorithm developed by Lindgren *et al.* (*J. Chemometrics*, **7**, 45 (1993)) is modified to provide two faster and more economical algorithms. The performances of these new algorithms are compared with that of De Jong and Ter Braak's (*J. Chemometrics*, **8**, 169 (1994)) modified kernel algorithm in terms of speed and the new algorithms are shown to be much faster. A very fast kernel algorithm for updating PLS models in a recursive manner and for exponentially discounting past data is also presented. © 1997 by John Wiley & Sons, Ltd.

KEY WORDS partial least squares (PLS); algorithms; faster kernel algorithms; recursive PLS; exponentially weighted PLS

1. INTRODUCTION

A PLS model can be computed using either the classical NIPALS algorithm or a kernel algorithm.[1] Other ways of computing PLS latent vectors are discussed by Hoskuldsson.[2] These are sequential algorithms where one latent vector is computed at a time and then the **X**- and **Y**-matrices are deflated to compute the next latent vector.

A typical PLS algorithm is as follows:

(i) Mean-center and scale the **X**- and **Y**-matrices;
(ii) Compute the quantities **w**, **t**, **q**, **u** and **p** using either the NIPALS algorithm or a kernel algorithm.
(iii) Deflate the **X**- and **Y**-matrices by subtracting the computed latent vectors from them:

$$\mathbf{X}_{a+1} = \mathbf{X}_a - \mathbf{t}_a\mathbf{p}_a^{\mathrm{T}} \quad (1)$$

$$\mathbf{Y}_{a+1} = \mathbf{Y}_a - \mathbf{t}_a\mathbf{q}_a^{\mathrm{T}} \quad (2)$$

(iv) Go to step (ii) to compute the next latent vector.

In PLS, generally both the **X**- and **Y**-matrices are deflated in step (iii) after each latent vector computation. However, Hoskuldsson[3] has shown that deflating the **Y**-matrix is

* Present address: GE Superabrasives, 6325 Huntley Road, PO Box 568, Worthington, OH, 43085,U.S.A.
‡ Corresponding author.

CCC 0886–9383/97/010073–13 $17.50

Received 31 January 1996
Accepted 21 August 1996

optional. Lindgren *et al.*[1] took advantage of this fact to make the kernel algorithm faster by deflating the **X**-matrix only. In this paper it is shown that only one of either the **X**- or the **Y**-matrix needs to be deflated. This result then leads to two new and very fast PLS kernel algorithms.

2. NOMENCLATURE

The following notation is used. Uppercase bold variables are matrices and lowercase bold variables are column vectors.

$\mathbf{X}$	predictor variables matrix $(N \times K)$
$\mathbf{Y}$	response variables matrix $(N \times M)$
$\mathbf{B}_{\mathrm{PLS}}$	PLS regression coefficients matrix $(K \times M)$
$\mathbf{W}$	PLS weights matrix for $\mathbf{X}$ $(K \times A)$
$\mathbf{P}$	PLS loadings matrix for $\mathbf{X}$ $(K \times A)$
$\mathbf{Q}$	PLS loadings matrix for $\mathbf{Y}$ $(M \times A)$
$\mathbf{R}$	PLS weights matrix to compute scores $\mathbf{T}$ directly from original $\mathbf{X}$ $(K \times A)$
$\mathbf{T}$	PLS scores matrix of $\mathbf{X}$ $(N \times A)$
$\mathbf{w}_a$	a column vector of $\mathbf{W}$
$\mathbf{p}_a$	a column vector of $\mathbf{P}$
$\mathbf{q}_a$	a column vector of $\mathbf{Q}$
$\mathbf{r}_a$	a column vector of $\mathbf{R}$
$\mathbf{t}_a$	a column vector of $\mathbf{T}$
K	number of X-variables
M	number of Y-variables
N	number of objects
A	number of components in PLS model
a	integer counter for latent variable dimension.

3. PROOF THAT ONLY ONE OF **X** OR **Y** NEEDS TO BE DEFLATED

To prove this result one must show that

$$\mathbf{X}_{a+1}^{\mathrm{T}}\mathbf{Y}_{a+1} = \mathbf{X}_{a}^{\mathrm{T}}\mathbf{Y}_{a+1} = \mathbf{X}_{a+1}^{\mathrm{T}}\mathbf{Y}_{a} \tag{3}$$

In PLS, $\mathbf{X}$ and $\mathbf{Y}$ are deflated as shown in equations (1) and (2). Using equations (1) and (2), $\mathbf{X}_{a+1}^{\mathrm{T}}\mathbf{Y}_{a+1}^{\mathrm{T}}$ can be expanded as

$$\begin{aligned}\mathbf{X}_{a+1}^{\mathrm{T}}\mathbf{Y}_{a+1}^{\mathrm{T}} &= (\mathbf{X}_a - \mathbf{t}_a\mathbf{p}_a^{\mathrm{T}})^{\mathrm{T}}(\mathbf{Y}_a - \mathbf{t}_a\mathbf{q}_a^{\mathrm{T}}) \\ &= \mathbf{X}_a^{\mathrm{T}}\mathbf{Y}_a - \mathbf{X}_a^{\mathrm{T}}\mathbf{t}_a\mathbf{q}_a^{\mathrm{T}} - \mathbf{p}_a\mathbf{t}_a^{\mathrm{T}}\mathbf{Y}_a + \mathbf{p}_a\mathbf{t}_a^{\mathrm{T}}\mathbf{t}_a\mathbf{q}_a^{\mathrm{T}} \\ &= \mathbf{X}_a^{\mathrm{T}}\mathbf{Y}_a - \mathbf{X}_a^{\mathrm{T}}\mathbf{t}_a\mathbf{q}_a^{\mathrm{T}} - \mathbf{p}_a\mathbf{t}_a^{\mathrm{T}}\mathbf{Y}_a + \mathbf{p}_a\mathbf{q}_a^{\mathrm{T}}\mathbf{t}_a^{\mathrm{T}}\mathbf{t}_a \end{aligned} \tag{4}$$

Note that $\mathbf{t}_a^{\mathrm{T}}\mathbf{t}_a$ is a scalar and therefore $\mathbf{p}_a(\mathbf{t}_a^{\mathrm{T}}\mathbf{t}_a)\mathbf{q}_a^{\mathrm{T}} = \mathbf{p}_a\mathbf{q}_a^{\mathrm{T}}(\mathbf{t}_a^{\mathrm{T}}\mathbf{t}_a)$.

The quantities $\mathbf{q}_a$ and $\mathbf{p}_a$ in PLS are computed as

$$\mathbf{q}_a = \frac{\mathbf{Y}_a^{\mathrm{T}}\mathbf{t}_a}{\mathbf{t}_a^{\mathrm{T}}\mathbf{t}_a} \tag{5}$$

$$\mathbf{p}_a = \frac{\mathbf{X}_a^{\mathrm{T}}\mathbf{t}_a}{\mathbf{t}_a^{\mathrm{T}}\mathbf{t}_a} \tag{6}$$

Rearranging equations (5) and (6),

$$\mathbf{t}_a^{\mathrm{T}}\mathbf{Y}_a = \mathbf{q}_a^{\mathrm{T}}(\mathbf{t}_a^{\mathrm{T}}\mathbf{t}_a) \tag{7}$$

$$\mathbf{X}_a^{\mathrm{T}}\mathbf{t}_a = (\mathbf{t}_a^{\mathrm{T}}\mathbf{t}_a)\mathbf{p}_a \tag{8}$$

I. Proof that deflation of $\mathbf{Y}$ is optional or that $\mathbf{X}_{a+1}^{\mathrm{T}}\mathbf{Y}_{a+1} = \mathbf{X}_{a+1}^{\mathrm{T}}\mathbf{Y}_a$: substituting equation (8) into equation (4),

$$\begin{aligned}\mathbf{X}_{a+1}^{\mathrm{T}}\mathbf{Y}_{a+1} &= \mathbf{X}_a^{\mathrm{T}}\mathbf{Y}_a - \mathbf{p}_a\mathbf{q}_a^{\mathrm{T}}(\mathbf{t}_a^{\mathrm{T}}\mathbf{t}_a) - \mathbf{p}_a\mathbf{t}_a^{\mathrm{T}}\mathbf{Y}_a + \mathbf{p}_a\mathbf{q}_a^{\mathrm{T}}(\mathbf{t}_a^{\mathrm{T}}\mathbf{t}_a)\\ &= \mathbf{X}_a^{\mathrm{T}}\mathbf{Y}_a - \mathbf{p}_a\mathbf{t}_a^{\mathrm{T}}\mathbf{Y}_a = (\mathbf{X}_a^{\mathrm{T}} - \mathbf{p}_a\mathbf{t}_a^{\mathrm{T}})\mathbf{Y}_a = (\mathbf{X}_a - \mathbf{t}_a\mathbf{p}_a^{\mathrm{T}})^{\mathrm{T}}\mathbf{Y}_a\\ &= \mathbf{X}_{a+1}^{\mathrm{T}}\mathbf{Y}_a\end{aligned} \tag{9}$$

II. Proof that deflation of $\mathbf{X}$ is optional or that $\mathbf{X}_{a+1}^{\mathrm{T}}\mathbf{Y}_{a+1} = \mathbf{X}_a^{\mathrm{T}}\mathbf{Y}_{a+1}$: substituting equation (7) into equation (4),

$$\begin{aligned}\mathbf{X}_{a+1}^{\mathrm{T}}\mathbf{Y}_{a+1} &= \mathbf{X}_a^{\mathrm{T}}\mathbf{Y}_a - \mathbf{X}_a^{\mathrm{T}}\mathbf{t}_a\mathbf{q}_a^{\mathrm{T}} - \mathbf{p}_a\mathbf{q}_a^{\mathrm{T}}(\mathbf{t}_a^{\mathrm{T}}\mathbf{t}_a) + \mathbf{p}_a\mathbf{q}_a{}^{\mathrm{T}}(\mathbf{t}_a^{\mathrm{T}}\mathbf{t}_a)\\ &= \mathbf{X}_a^{\mathrm{T}}\mathbf{Y}_a - \mathbf{X}_a^{\mathrm{T}}\mathbf{t}_a\mathbf{q}_a^{\mathrm{T}}\\ &= \mathbf{X}_a^{\mathrm{T}}(\mathbf{Y}_a - \mathbf{t}_a\mathbf{q}_a^{\mathrm{T}})\\ &= \mathbf{X}_a^{\mathrm{T}}\mathbf{Y}_{a+1}\end{aligned} \tag{10}$$

4. NIPALS ALGORITHM

In the NIPALS algorithm the computational effort for deflating $\mathbf{X}$ and $\mathbf{Y}$ is minimal compared with the iterative process of computing the latent vectors, the exception being when there is only a single response variable ($\mathbf{y}$). In this latter case only two iterations are needed for the NIPALS algorithm to converge for each latent vector and therefore a significant proportion of the computational effort is spent on deflating $\mathbf{X}$ and $\mathbf{y}$. Since only the $(N \times 1)$ $\mathbf{y}$-vector needs to be deflated after each latent vector computation, the speed of the NIPALS algorithm is improved. The MATLAB® code for the modified NIPALS algorithm is given in the Appendix.

5. KERNEL ALGORITHM

The kernel algorithm was proposed by Lindgren *et al.*[1] and is given below. In the remainder of the paper it is assumed that the columns of $\mathbf{X}$ and $\mathbf{Y}$ are mean-centered and scaled appropriately prior to PLS model estimation using a kernel algorithm.

1. Compute the covariance matrices $\mathbf{X}^{\mathrm{T}}\mathrm{X}$ and $\mathbf{X}^{\mathrm{T}}\mathbf{Y}$. The kernel matrix $\mathbf{X}^{\mathrm{T}}\mathbf{Y}\mathbf{Y}^{\mathrm{T}}\mathbf{X}$ can be computed by multiplication of $\mathbf{X}^{\mathrm{T}}\mathbf{Y}$ by $(\mathbf{X}^{\mathrm{T}}\mathbf{Y})^{\mathrm{T}}$.
2. The PLS weight vector $\mathbf{w}_a$ is computed as the eigenvector corresponding to the largest eigenvalue of $(\mathbf{X}^{\mathrm{T}}\mathbf{Y}\mathbf{Y}^{\mathrm{T}}\mathbf{X})_a$ using either the power method or other approaches (SVD, etc.):

$$\mathbf{w}_a \propto (\mathbf{X}^{\mathrm{T}}\mathbf{Y}\mathbf{Y}^{\mathrm{T}}\mathbf{X})_a\mathbf{w}_a \tag{11}$$

3. The PLS loading vectors $\mathbf{p}_a$ and $\mathbf{q}_a$ are computed as

$$\mathbf{p}_a^{\mathrm{T}} = \frac{\mathbf{w}_a^{\mathrm{T}}(\mathbf{X}^{\mathrm{T}}\mathbf{X})_a}{\mathbf{w}_a^{\mathrm{T}}(\mathbf{X}^{\mathrm{T}}\mathbf{X})_a\mathbf{w}_a} \tag{12}$$

$$\mathbf{q}_a^{\mathrm{T}} = \frac{\mathbf{w}_a^{\mathrm{T}}(\mathbf{X}^{\mathrm{T}}\mathbf{Y})_a}{\mathbf{w}_a^{\mathrm{T}}(\mathbf{X}^{\mathrm{T}}\mathbf{X})_a\mathbf{w}_a} \tag{13}$$

4. After each latent vector computation the covariance matrices $\mathbf{X}^T\mathbf{X}$ and $\mathbf{X}^T\mathbf{Y}$ can be updated as

$$(\mathbf{X}^T\mathbf{X})_{a+1} = (\mathbf{I} - \mathbf{w}_a\mathbf{p}_a^T)^T(\mathbf{X}^T\mathbf{X})_a(\mathbf{I} - \mathbf{w}_a\mathbf{p}_a^T) \tag{14}$$

$$(\mathbf{X}^T\mathbf{Y})_{a+1} = (\mathbf{I} - \mathbf{w}_a\mathbf{p}_a^T)^T(\mathbf{X}^T\mathbf{Y})_a \tag{15}$$

Note that only the **X**-matrix is being deflated in the above equations.

5.1. Modification to kernel algorithm

De Jong and Ter Braak[4] proposed a modification to step 4 of the kernel algorithm of Lindgren *et al.*[1] whereby the deflation of $\mathbf{X}^T\mathbf{X}$ and $\mathbf{X}^T\mathbf{Y}$ by expansion of equations (14) and (15) is simplified to

$$(\mathbf{X}^T\mathbf{X})_{a+1} = (\mathbf{X}^T\mathbf{X})_a - \mathbf{p}_a\mathbf{p}_a^T(\mathbf{t}_a^T\mathbf{t}_a) \tag{16}$$

$$(\mathbf{X}^T\mathbf{Y})_{a+1} = (\mathbf{X}^T\mathbf{Y})_a - \mathbf{p}_a\mathbf{q}_a^T(\mathbf{t}_a^T\mathbf{t}_a) \tag{17}$$

This reduces the computational effort during the deflation step by avoiding the multiplication of $\mathbf{X}^T\mathbf{X}$ and $\mathbf{X}^T\mathbf{Y}$ by $(\mathbf{I} - \mathbf{w}_a\mathbf{p}_a{}^T)$. This modified kernel algorithm was proven to be much faster than the original kernel algorithm.

5.2. Further modification to kernel algorithm

As shown earlier, only one of either the *X*- or the **Y**-matrix needs to be deflated. Therefore in the kernel logarithm one does not need to deflate **X**. The only deflation necessary is the deflation of **Y** in $\mathbf{X}^T\mathbf{Y}$:

$$(\mathbf{X}^T\mathbf{Y})_{a+1} = (\mathbf{X}^T\mathbf{Y})_a - \mathbf{X}_a^T\mathbf{t}_a\mathbf{q}_a^T \tag{18}$$

Substituting equation (8) into the above equation,

$$(\mathbf{X}^T\mathbf{Y})_{a+1} = (\mathbf{X}^T\mathbf{Y})_a - \mathbf{p}_a\mathbf{q}_a^T(\mathbf{t}_a^T\mathbf{t}_a) \tag{19}$$

Note that this equation is same as equation (17). If **X** is not being deflated, then equations (12) and (13) in step 3 of the kernel algorithm need to be modified so that the loading vectors $\mathbf{p}_a$ and $\mathbf{q}_a$ can be computed using the non-deflated or original **X**-matrix. The score vectors **T** can be directly computed from the original **X** by the equation

$$\mathbf{T} = \mathbf{XR}$$

$$[\mathbf{t}_1\ \mathbf{t}_2\ \ldots\ \mathbf{t}_A] = \mathbf{X}[\mathbf{r}_1\ \mathbf{r}_2\ \ldots\ \mathbf{r}_A] \tag{20}$$

where **R** is given by

$$\mathbf{R} = \mathbf{W}(\mathbf{P}^T\mathbf{W})^{-1} \tag{21}$$

The columns of **R** can be easily computed sequentially. A sequential relationship to compute **R** is derived as

$$\begin{aligned}
&\mathbf{t}_1 = \mathbf{X}_1\mathbf{w}_1 = \mathbf{X}\mathbf{w}_1 \\
&\mathbf{t}_2 = \mathbf{X}_2\mathbf{w}_2 = \mathbf{X}(\mathbf{I} - \mathbf{w}_1\mathbf{p}_1^T)\mathbf{w}_2 \\
&\vdots \\
&\mathbf{t}_A = \mathbf{X}(\mathbf{I} - \mathbf{w}_1\mathbf{p}_1^T)(\mathbf{I} - \mathbf{w}_2\mathbf{p}_2^T)\cdots(\mathbf{I} - \mathbf{w}_{A-1}\mathbf{p}_{A-1}^T)\mathbf{w}_A
\end{aligned} \tag{22}$$

Therefore

$$\begin{aligned}
&\mathbf{r}_1 = \mathbf{w}_1 \\
&\mathbf{r}_2 = (\mathbf{I} - \mathbf{w}_1\mathbf{p}_1^{\mathrm{T}})\mathbf{w}_2 \\
&\vdots \\
&\mathbf{r}_A = (\mathbf{I} - \mathbf{w}_1\mathbf{p}_1^{\mathrm{T}})(\mathbf{I} - \mathbf{w}_2\mathbf{p}_2^{\mathrm{T}}) \cdots (\mathbf{I} - \mathbf{w}_{A-1}\mathbf{p}_{A-1}^{\mathrm{T}})\mathbf{w}_A
\end{aligned} \tag{23}$$

The above relationship can be represented as

$$\mathbf{r}_i = \mathbf{B}_i\mathbf{w}_i \tag{24}$$

where $\mathbf{B}$ is computed recursively using[5]

$$\mathbf{B}_{i+1} = \mathbf{B}_i(\mathbf{I} - \mathbf{w}_i\mathbf{p}_i{}^{\mathrm{T}}) = \mathbf{B}_i - \mathbf{B}_i\mathbf{w}_i\mathbf{p}_i^{\mathrm{T}} = \mathbf{B}_i - \mathbf{r}_i\mathbf{p}_i{}^{\mathrm{T}} \tag{25}$$

$$\mathbf{B}_1 = \mathbf{I} \tag{26}$$

Computing $\mathbf{r}_i$ using the above equations involves maintaining a $K \times K$ matrix $\mathbf{B}_i$ and multiplication of this matrix by $\mathbf{w}_i$ in equation (24), which could be very computationally intense with a higher number of X-variables. This can be avoided by computing $\mathbf{r}_i$ using the recursive relationship

$$\begin{aligned}
&\mathbf{r}_1 = \mathbf{w}_1 \\
&\mathbf{r}_i = \mathbf{w}_i - \mathbf{p}_1^{\mathrm{T}}\mathbf{w}_i\mathbf{r}_1 - \mathbf{p}_2^{\mathrm{T}}\mathbf{w}_i\mathbf{r}_2 - \cdots - \mathbf{p}_{i-1}^{\mathrm{T}}\mathbf{w}_i\mathbf{r}_{i-1}, \quad i > 1
\end{aligned} \tag{27}$$

The major rate-determining step in the kernel algorithm in terms of computational effort is construction of the matrices $\mathbf{X}^{\mathrm{T}}\mathbf{X}$ and $\mathbf{X}^{\mathrm{T}}\mathbf{Y}$. In this modified kernel algorithm, construction of $\mathbf{X}^{\mathrm{T}}\mathbf{X}$ is not necessary. If the number of data points in $\mathbf{X}$ is much greater than the number of variables in $\mathbf{X}$ ($N >> K$), then it might be convenient to compute $\mathbf{X}^{\mathrm{T}}\mathbf{X}$, since storing $\mathbf{X}^{\mathrm{T}}\mathbf{X}$ would require less memory space than storing $\mathbf{X}$. However, if one is not restricted by the computer memory, then not computing $\mathbf{X}^{\mathrm{T}}\mathbf{X}$ and directly using $\mathbf{X}$ in the kernel algorithm to compute $\mathbf{p}$ and $\mathbf{q}$ would require less computational effort.

In this paper, two new modified kernel algorithms are developed. In the first algorithm, algorithm #1, the covariance matrix $\mathbf{X}^{\mathrm{T}}\mathbf{X}$ is not computed and $\mathbf{X}$ is used directly in computations for $\mathbf{p}_a$ and $\mathbf{q}_a$ as shown in step 4(a) (equations (31)–(33)). In the second modified algorithm, algorithm #2, the covariance matrix $\mathbf{X}^{\mathrm{T}}\mathbf{X}$ is computed once and then used subsequently in the computations for $\mathbf{p}_a$ and $\mathbf{q}_a$ as shown in step 4(b) (equations (34)–(35)). The new kernel algorithms are presented below. Algorithms #1 and #2 differ only in steps 1 and 4.

1. Compute the covariance matrix $\mathbf{X}^{\mathrm{T}}\mathbf{Y}$. Computation of $\mathbf{X}^{\mathrm{T}}\mathbf{X}$ is optional.
2. If there are fewer Y-variables, then compute $\mathbf{q}_a$ as the eigenvector corresponding to the largest eigenvalue of $(\mathbf{Y}^{\mathrm{T}}\mathbf{X}^{\mathrm{T}}\mathbf{X}\mathbf{Y})_a$ and $\mathbf{w}_a$ can be computed from the relationships

$$\mathbf{w}_a{}^{\mathrm{T}} = (\mathbf{X}^{\mathrm{T}}\mathbf{Y})_a\mathbf{q}_a \tag{28}$$

$$\mathbf{w}_a = \frac{\mathbf{w}_a}{|\mathbf{w}_a|} \tag{29}$$

   If there are fewer X-variables, then compute $\mathbf{w}_a$ as the eigenvector corresponding to the largest eigenvalue of $\mathbf{X}^{\mathrm{T}}\mathbf{Y}\mathbf{Y}^{\mathrm{T}}\mathbf{X}$ as shown in equation (11).
3. Compute $\mathbf{r}_a$:

$$\begin{aligned}
&\mathbf{r}_1 = \mathbf{w}_1 \\
&\mathbf{r}_a = \mathbf{w}_a - \mathbf{p}_1^{\mathrm{T}}\mathbf{w}_a\mathbf{r}_1 - \mathbf{p}_2{}^{\mathrm{T}}\mathbf{w}_a\mathbf{r}_2 - \cdots - \mathbf{p}_{a-1}^{\mathrm{T}}\mathbf{w}_a\mathbf{r}_{a-1}, \quad a > 1
\end{aligned} \tag{30}$$

4. Compute the loading vectors $\mathbf{p}_a$ and $\mathbf{q}_a$ according to either of the following modified algorithms.

   (a) *Modified algorithm #1*. The covariance matrix $\mathbf{X}^T\mathbf{X}$ is not constructed, but rather $\mathbf{X}$ is used directly in the computations as

$$\mathbf{t}_a = \mathbf{X}\mathbf{r}_a \tag{31}$$

$$\mathbf{p}_a = \frac{\mathbf{t}_a^T\mathbf{X}}{\mathbf{t}_a^T\mathbf{t}_a} \tag{32}$$

$$\mathbf{q}_a^T = \frac{\mathbf{r}_a^T(\mathbf{X}^T\mathbf{Y})}{\mathbf{t}_a^T\mathbf{t}_a} \tag{33}$$

   (b) *Modified algorithm #2*. The covariance matrix $\mathbf{X}^T\mathbf{X}$ is computed only once with the original $\mathbf{X}$. Then in all dimensions

$$\mathbf{p}_a^T = \frac{\mathbf{r}_a^T(\mathbf{X}^T\mathbf{X})}{\mathbf{r}_a^T(\mathbf{X}^T\mathbf{X})\mathbf{r}_a} \tag{34}$$

$$\mathbf{q}_a^T = \frac{\mathbf{r}_a^T(\mathbf{X}^T\mathbf{Y})_a}{\mathbf{r}_a^T(\mathbf{X}^T\mathbf{X})\mathbf{r}_a} \tag{35}$$

5. Update the covariance matrix $\mathbf{X}^T\mathbf{Y}$:

$$(\mathbf{X}^T\mathbf{Y})_{a+1} = (\mathbf{X}^T\mathbf{Y})_a - \mathbf{p}_a\mathbf{q}_a^T(\mathbf{t}_a^T\mathbf{t}_a) \tag{36}$$

6. Store $\mathbf{w}$, $\mathbf{p}$, $\mathbf{q}$ and $\mathbf{r}$ in $\mathbf{W}$, $\mathbf{P}$, $\mathbf{Q}$ and $\mathbf{R}$:

$$\mathbf{W} = [\mathbf{w}_1\ \mathbf{w}_2\ \cdots\ \mathbf{w}_A],\quad \mathbf{P} = [\mathbf{p}_1\ \mathbf{p}_2\ \cdots\ \mathbf{p}_A],\quad \mathbf{Q} = [\mathbf{q}_1\ \mathbf{q}_2\ \cdots\ \mathbf{q}_A],\quad \mathbf{R} = [\mathbf{r}_1\ \mathbf{r}_2\ \cdots\ \mathbf{r}_A] \tag{37}$$

7. Go to step 2 for the next latent vector computation.
8. When done computing latent vectors, the regression coefficients for the PLS model are given by

$$\mathbf{B}_{\mathrm{PLS}} = \mathbf{R}\mathbf{Q}^T \tag{38}$$

The MATLAB® code for both modified algorithms is given in the Appendix.

6. COMPARISON OF KERNEL ALGORITHMS

Here the new kernel algorithms are compared with De Jong and Ter Braak's kernel algorithm in terms of speed. All kernel algorithms provided the same PLS models as the NIPALS algorithm.

Table 1. Design variables for speed comparison of various kernel algorithms

Design variables	Low	Center	High
Number of data points (N)	500	750	1000
Number of X-Variables (K)	10	20	30
Number of Y-variables (M)	1	3	5
Number of latent vectors (A)	3	4	5

Table 2. 2^4 factorial design for comparison of various kernel algorithms

Run number	Number of data points (N)	Number of X-variables (K)	Number of Y-variables (M)	Number of latent vectors (A)
1	500	10	5	3
2	500	10	5	5
3	500	10	1	3
4	500	10	1	5
5	500	30	5	3
6	500	30	5	5
7	500	30	1	3
8	500	30	1	5
9	750	20	3	4
10	1000	10	5	3
11	1000	10	5	5
12	1000	10	1	3
13	1000	10	1	5
14	1000	30	5	3
15	1000	30	5	5
16	1000	30	1	3
17	1000	30	1	5

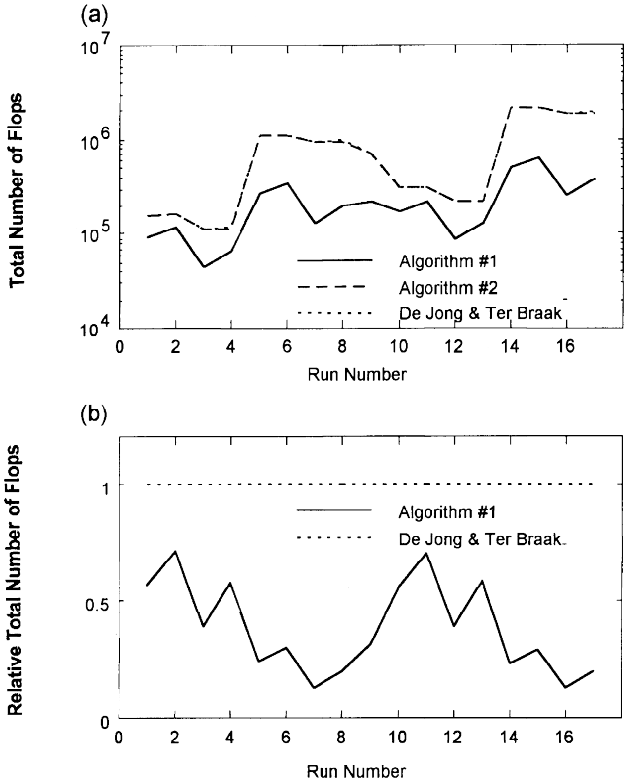

Figure 1. Comparison of various kernel algorithms: (a) total number of flops; (b) relative total number of flops

Since De Jong and Ter Braak's kernel algorithm has been proven to be faster than the original kernel algorithm of Lindgren *et al.*,[1] the original algorithm is not included in this comparison study. The comparison criterion used is the floating point operations (flops) used in MATLAB® to compute the PLS model. This criterion is the same as that of De Jong and Ter Braak[4] when they compared their kernel algorithm with the original kernel algorithm.

The algorithms were tested for speed to variations in the following four design variables: (i) number of data points (N), (ii) number of X-variables (K), (iii) number of Y-variables (M) and (iv) number of latent vectors (A) used in the PLS model. The low and high levels of these designed variables are given in Table 1. The 2^4 factorial design augmented with a center point and shown in Table 2 was performed.

The performance results of the three kernel algorithms are plotted in Figure 1. In Figure 1(a) the total number of flops consumed is plotted against the run number of the factorial design, while in Figure 1(b) the relative flops (i.e. the total number of flops for the given method divided by the total number of flops for the De Jong and Ter Braak method) are shown. The total flops consumed are the combined flops used for constructing the covariance matrices and computing the latent vectors. As is evident from Figure 1(a), algorithm #1, where $\mathbf{X}^T\mathbf{X}$ is not computed, is much faster than algorithm #2 and the De Jong and Ter Braak algorithm. In Figure 1(b) it is seen to consume as little as one-fifth of the flops consumed by the De Jong and Ter Braak algorithm as the number of X-variables increases. There is no apparent difference

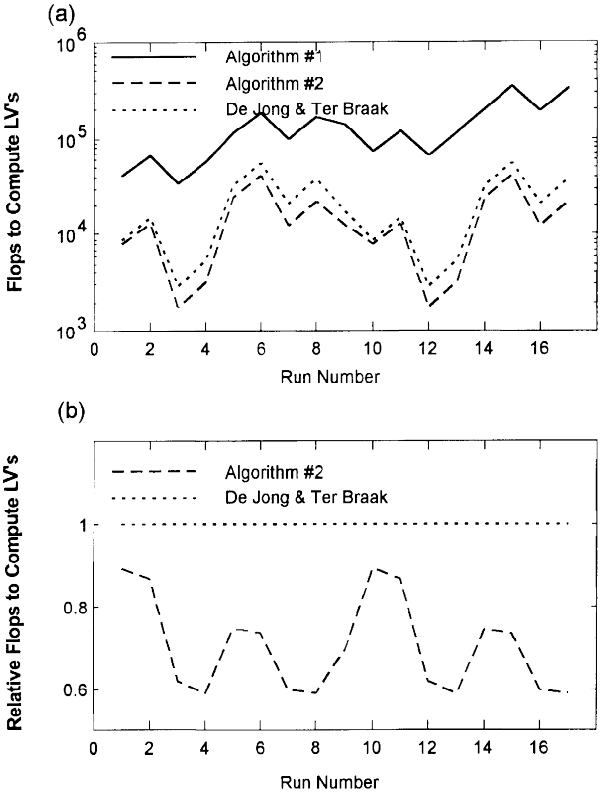

Figure 2. Comparison of various kernel algorithms: (a) flops required to compute latent vectors; (b) relative flops required to compute latent vectors

between algorithm #2 and the De Jong and Ter Braak algorithm, since most of the flops consumed in these algorithms are for construction of $\mathbf{X}^T\mathbf{X}$ and $\mathbf{X}^T\mathbf{Y}$.

In Figure 2 the flops consumed for computing only the latent vectors of the PLS model (after the covariance matrices have been calculated) are plotted against the run number of the factorial design for each of the three kernel algorithms. In this case, algorithm #2 provides the best results. Algorithm #1 requires the highest number of flops because it uses $\mathbf{X}$ (which is larger in dimension than $\mathbf{X}^T\mathbf{X}$) directly in the computations for $\mathbf{p}_a$ and $\mathbf{q}_a$. The relative flops for algorithm #2 and the De Jong and Ter Braak algorithm are shown in Figure 2(b). As is evident, algorithm #2 is much faster than the De Jong and Ter Braak algorithm as the number of *X*-variables increase.

7. DISCUSSION

In this section we discuss the modified algorithms that would be advantageous to use under different circumstances.

7.1 Missing data

Rannar *et al.*[6] discussed how to handle missing data with the original kernel algorithm of Lindgren *et al.*[1] They proposed a method along the lines of the EM algorithm wherein the missing data are replaced with some initial values (i.e. column or row mean, etc.). A PLS model is then calculated and the missing data are replaced by their PLS estimates. This procedure is repeated until some convergence criterion is satisfied. The problem they found with this procedure was that after replacing the missing data with PLS estimates, the new covariance matrices need to be recomputed. To overcome this problem, Rannar *et al.*[6] proposed a so-called reduced EM algorithm wherein the replacement of the missing data is accomplished by updating but not recomputing the covariance matrices. This gives a faster but less precise algorithm. As discussed earlier, the construction of $\mathbf{X}^T\mathbf{X}$ is the most computationally intense task in the kernel algorithm. Since the modified algorithm #1 does not require the computation of $\mathbf{X}^T\mathbf{X}$, it can be used with the full EM algorithm for handling missing data without sacrificing precision and speed and is therefore recommended when one has missing data.

7.2. Cross-validation

If one is not sure how many latent vectors would be sufficient for a PLS model, then one can use the cross-validation (CV) technique to select the optimal number of latent vectors. In this situation it might be beneficial to invest in a one-time construction of $\mathbf{X}^T\mathbf{X}$ rather than using the $\mathbf{X}$-matrix directly in the computations for $\mathbf{p}_a$ and $\mathbf{q}_a$, since these latter vectors would have to be recomputed many times in any CV method. Therefore modified algorithm #2, which is faster for computing latent vectors once $\mathbf{X}^T\mathbf{X}$ is available, is recommended for cross-validation.

7.3. Recursive exponentially weighted PLS

Very often new data are being collected on a regular basis and it is desirable to recursively update the PLS model using each new multivariate object as it becomes available. Furthermore, the process may be slowly changing with time and one would like to weight recent data more

heavily and discount past data in an exponentially weighted manner. Such situations arise frequently in the area of adaptive process control and calibration updating.

A fast kernel algorithm for recursive exponentially weighted updating of a PLS regression model is obtained using algorithm #2 with the covariance updating equations

$$(\mathbf{X}^{\mathrm{T}}\mathbf{X})_t = \lambda_t(\mathbf{X}^{\mathrm{T}}\mathbf{X})_{t-1} + \mathbf{x}_t^{\mathrm{T}}\mathbf{x}_t \tag{39}$$

$$(\mathbf{X}^{\mathrm{T}}\mathbf{Y})_t = \lambda_t(\mathbf{X}^{\mathrm{T}}\mathbf{Y})_{t-1} + \mathbf{x}_t^{\mathrm{T}}\mathbf{y}_t \tag{40}$$

Here $\mathbf{x}_t$ and $\mathbf{y}_t$ are the new object vectors observed at time t and $(\mathbf{X}^{\mathrm{T}}\mathbf{X})_t$ and $(\mathbf{X}^{\mathrm{T}}\mathbf{Y})_t$ are the updated covariance matrices at time t. At each new sampling period the previous data in the covariance matrices are being exponentially discounted with a forgetting factor λ_t $(0 < \lambda_t \leq 1)$ and the new data are being added. For $\lambda_t = 1$ no discounting of past data is done. Since the covariance matrices in equations (39) and (40) are updated with very little computational effort, algorithm #2 will be extremely fast in these applications. Dayal[7] and Dayal and MacGregor[8] have applied this algorithm to the adaptive multivariable control of a simulated continuous stirred tank reactor and to the updating of an on-line multi-output prediction model for an industrial mineral flotation circuit.

Wold[9] has recently published exponentially weighted algorithms for PCA and PLS. His algorithms are not recursive in that the entire expanded data set $(\mathbf{X}, \mathbf{Y})$ is used to build the model every time a new object becomes available rather than just recursively updating the previous covariance matrices with new objects as in equations (39) and (40). Wold's use of the NIPALS algorithm at each stage also makes his approach slower than the fast kernel algorithms proposed here. If one wishes to obtain exponentially weighted updated estimates of individual scores and loading vectors, then some form of stabilization method similar to those proposed by Wold[9] would have be incorporated into our algorithm as well in order to prevent unwarranted rotations in the latent vector space. However, such rotations are of no concern if one is only interested in an updated regression model.

Helland *et al.*[10] presented a recursive but not exponentially weighted algorithm for PLS. In their algorithm the old data in $\mathbf{X}$ and $\mathbf{Y}$ are captured by their loading matrices (i.e. $\mathbf{P}^{\mathrm{T}}$ and $\mathbf{Q}^{\mathrm{T}}$; the score vectors $\mathbf{t}$ and $\mathbf{u}$ are scaled to unit variance) and new data are added to these loading matrices. Although this algorithm keeps the size of the matrices which are used for PLS model calculations constant, it is still very slow compared with the new recursive exponentially weighted PLS kernel algorithm proposed here.

8. CONCLUSIONS

In PLS, generally both the $\mathbf{X}$- and $\mathbf{Y}$-matrices are deflated after each latent vector computation. In this paper we give a proof that only one of either the $\mathbf{X}$- or the $\mathbf{Y}$-matrix in PLS algorithms needs to be deflated. Using this proof, the original kernel algorithm of Lindgren *et al.*[1] is modified to develop two new faster and more economical algorithms. In the first algorithm the covariance matrix $\mathbf{X}^{\mathrm{T}}\mathbf{X}$ is not computed and $\mathbf{X}$ is used directly in computations for $\mathbf{p}$ and $\mathbf{q}$. In the second algorithm $\mathbf{X}^{\mathrm{T}}\mathbf{X}$ is computed once and then used in all subsequent computations for $\mathbf{p}$ and $\mathbf{q}$. The performances of these new algorithms are compared with that of De Jong and Ter Braak's algorithm in terms of speed and the new algorithms are shown to be much faster. Each of these new algorithms is shown to have certain advantages when performing cross-validation or treating missing data, and one of the algorithms provides a fast kernel algorithm for updating PLS models in a recursive manner and for exponentially discounting past data.

APPENDIX

Modified NIPALS algorithm

```
function [W,P,Q,R,beta]=nipals_y(X,Y,A)   % A=number of PLS latent vectors to be computed
                                          % beta=PLS regression coefficients
convcrit=1e-10;                           % convergence criteria for the NIPALS algorithm
for i=1:A,
  u=Y(:,find(std(Y)==max(std(Y))));       % assigning the y-variables with the largest variance
                                          % to u_initial
  diff=1;
  while (diff>convcrit),
    u0=u;
    w=X'*u/(u'*u);                        % compute w
    w=w/(sqrt(sum(w.^2)));                % normalizing w to unity
    r=w;                                  % compute r via equation (30) so that the score
    if i>1,                               % vector t can be computed directly from the
      for j=1:i-1,                        % original X
        r=r-(P(:,j)'*w)*R(:,j);
      end
    end
    t=X*r;                                % compute t
    q=Y'*t/(t'*t);                        % compute q
    u=Y*q/(q'*q);                         % compute u
    diff=(sqrt(sum((u0-u).^2)))/(sqrt(sum(u.^2)));
  end
  p=X*t/(t'*t);                           % compute X-loadings
  W=[W  w];                               % storing loadings and weights
  R=[R  r];
  P=[P  p];
  Q=[Q  q];
  T=[T  t];
  U=[U  u];
  Y=Y-t*q';                               % only Y block is deflated
end
  beta=R*Q';                              % compute the regression coefficients
end
```

The above modified NIPALS algorithm differs from the original only in the following steps: (i) **X** is not deflated; (ii) the additional vector **r** is computed according to equation (30); (iii) **t** is computed using **r** rather than **w**.

Modified kernel algorithm

```
XY=X'*Y;                                  % compute X^TY
for i=1:A,                                % A=number of PLS components to be computed
  if M==1,                                % if there is a single response variable, compute the
    w=XY;                                 % X-weights as shown here
```

```
   else                                          % else
      [C,D]=eig(XY'*XY);                         % first compute the eigenvectors of Y^T XX^T Y
      q=C(:,find(diag(D)==max(diag(D))));        % find the eigenvector corresponding to the largest
                                                 % eigenvalue
      w=(XY*q);                                  % compute X-weights
   end
   w=w/sqrt(w'*w);                               % normalize w to unity
   r=w;                                          % loop to compute r_i
   for j=1:i-1,
      r=r-(P(:,j)'*w)*R(:,j);
   end
   t=X*r;                                        % compute score vector
   tt=(t'*t);                                    % compute t^T t
   p=(X'*t)/tt;                                  % X-loadings
   q=(r'*XY)'/tt;                                % Y-loadings
   XY=XY-(p*q')*tt;                              % X^T Y deflation
   W=[W  w];                                     % store loadings and weights
   P=[P  p];
   Q=[Q  q];
   R=[R  r];
end
beta=R*Q';                                       % compute the regression coefficients
```

Modified kernel algorithm #2

```
XY=X'*Y;                                         % compute the covariance
XX=X'*X;                                         % matrices
for i=1:A,                                       % A=number of PLS components to be computed
   if M==1,                                      % if there is a single response variable, compute the
      w=XY;                                      % X-weights as shown here
   else                                          % else
      [C,D]=eig(XY'*XY);                         % first compute the eigenvectors of Y^T XX^T X
      q=C(:,find(diag(D)==max(diag(D))));        % find the eigenvector corresponding to the largest
                                                   eigenvalue
      w=(XY*q);                                  % compute X-weights
   end
   w=w/sqrt(w'*w);                               % normalize w to unity
   r=w;                                          % loop to compute r_i
   for j=l:i-1,
      r=r-(P(:,j)'*w)*R(:,j);
   end
   tt=(r'*XX*r);                                 % compute t^T t
   p=(r'*XX)'/tt;                                % X-loadings
   q=(r'*XY)'/tt;                                % Y-loadings
   XY=XY-(p*q')*tt;                              % X^T Y deflation
   W=[W  w];                                     % storing loadings and weights
   P=[P  P];
   Q=[Q  q];
```

```
  R=[R  r];
end
beta=R*Q';                    % compute the regression coefficients
```

REFERENCES

1. F. Lindgren, P. Geladi and S. Wold, *J. Chemometrics*, 7, 45 (1993).
2. A. Hoskuldsson, *J. Chemometrics*, **9**, 91 (1995).
3. A. Hoskuldsson, *J. Chemometrics*, **2**, 211 (1988).
4. S. De Jong and C. J. F. Ter Braak, *J. Chemometrics*, **8**, 169 (1994).
5. A.Hoskuldsson, *Chemometrics Intell. Lab. Syst.* **14**, 139 (1992).
6. S. Rannar, P. Geladi, F. Lindgren and S. Wold, *J. Chemometrics*, **9**, 459 (1995).
7. B. S. Dayal, *PhD. Thesis*, Department of Chemical Engineering, McMaster University, Hamilton, Ont. (1996).
8. B. S. Dayal and J. F. MacGregor, *J. Process Control*, in press (1996).
9. S. Wold, *Chemometrics Intell Lab. Syst.* **23**, 149 (1994).
10. K. Helland, H. E. Berntsen, O. S. Borgen and H. Marten, *Chemometrics Intell. Lab. Syst.* **14**, 129 (1991).